Interrupt Service Routines (ISRs)

Feb 28, 2026

Simple Example of the Benefit of ISRs

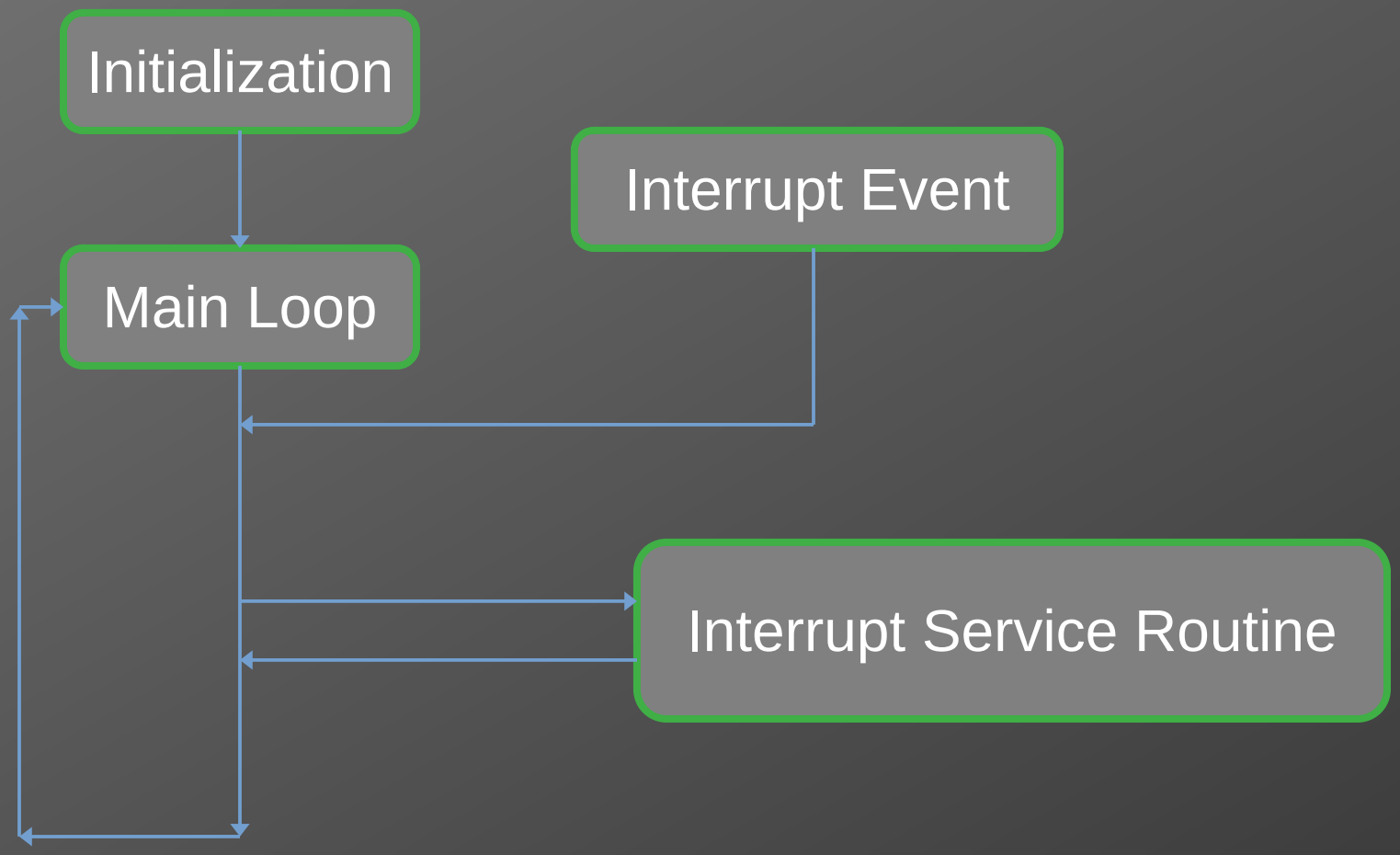
100% CPU Utilization

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(500);
  digitalWrite(LED_BUILTIN, LOW);
  delay(500);
}
```

~0% Utilization

```
#include <Arduino.h>

const int LED_PIN = 25;
const int D14_PIN = 14;

volatile bool ledState = false;
bool repeating_timer_callback(struct repeating_timer *t) {
    digitalWrite(D14_PIN, HIGH);
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState);
    digitalWrite(D14_PIN, LOW);
    return true; // Return true to keep the timer running
}

struct repeating_timer timer;

void setup() {
    pinMode(LED_PIN, OUTPUT);
    pinMode(D14_PIN, OUTPUT);
    add_repeating_timer_ms(-500, repeating_timer_callback, NULL, &timer);
}

void loop() {

}
```

How do ISRs Work?



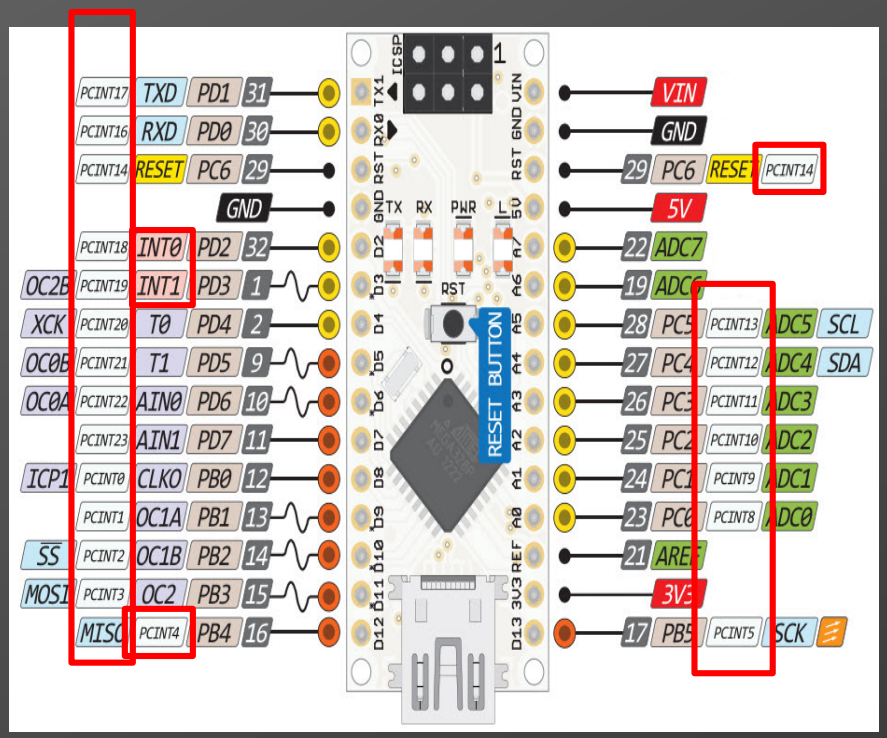
What Can Cause Interrupts?

Higher Priority

11.1 Interrupt Vectors in ATmega328P

Table 11-1. Reset and Interrupt Vectors in ATmega328P

Vector No.	Program Address	Source	Interrupt Definition
1	0x0000	RESET	External pin, power-on reset, brown-out reset and watchdog system reset
2	0x002	INT0	External interrupt request 0
3	0x0004	INT1	External interrupt request 1
4	0x0006	PCINT0	Pin change interrupt request 0
5	0x0008	PCINT1	Pin change interrupt request 1
6	0x000A	PCINT2	Pin change interrupt request 2
7	0x000C	WDT	Watchdog time-out interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 compare match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 compare match B
10	0x0012	TIMER2 OVF	Timer/Counter2 overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 capture event
12	0x0016	TIMER1 COMPA	Timer/Counter1 compare match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 compare match B
14	0x001A	TIMER1 OVF	Timer/Counter1 overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 compare match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 compare match B
17	0x0020	TIMER0 OVF	Timer/Counter0 overflow
18	0x0022	SPI, STC	SPI serial transfer complete
19	0x0024	USART, RX	USART Rx complete
20	0x0026	USART, UDRE	USART, data register empty
21	0x0028	USART, TX	USART, Tx complete
22	0x002A	ADC	ADC conversion complete
23	0x002C	EE READY	EEPROM ready
24	0x002E	ANALOG COMP	Analog comparator
25	0x0030	TWI	2-wire serial interface
26	0x0032	SPM READY	Store program memory ready



What Can Cause Interrupts?

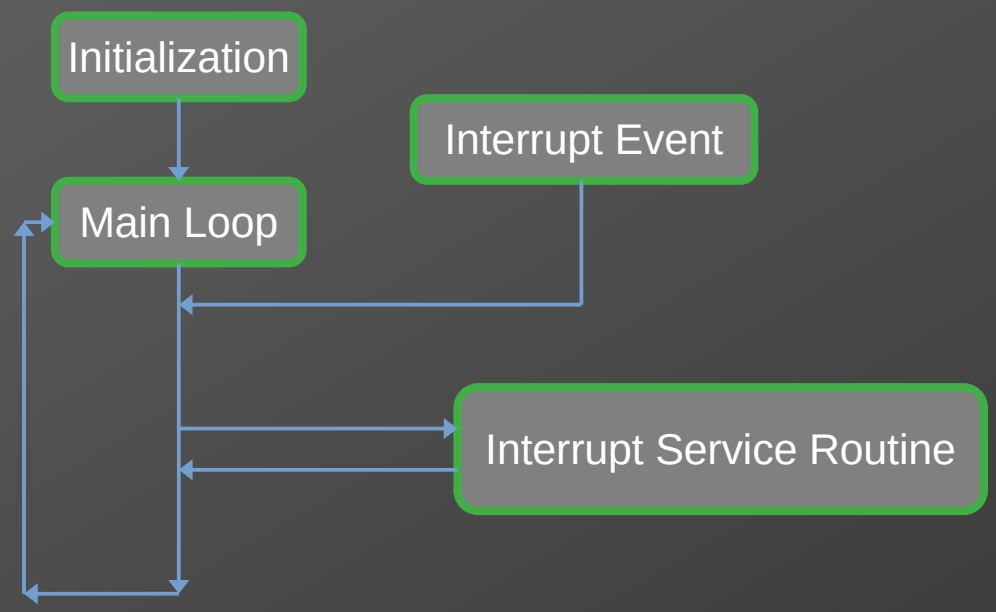
RP2040 (Pi Pico)

IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source	IRQ	Interrupt Source
0	TIMER_IRQ_0	6	XIP_IRQ	12	DMA_IRQ_1	18	SPI0_IRQ	24	I2C1_IRQ
1	TIMER_IRQ_1	7	PIO0_IRQ_0	13	IO_IRQ_BANK0	19	SPI1_IRQ	25	RTC_IRQ
2	TIMER_IRQ_2	8	PIO0_IRQ_1	14	IO_IRQ_QSPI	20	UART0_IRQ		
3	TIMER_IRQ_3	9	PIO1_IRQ_0	15	SIO_IRQ_PROC0	21	UART1_IRQ		
4	PWM_IRQ_WRAP	10	PIO1_IRQ_1	16	SIO_IRQ_PROC1	22	ADC_IRQ_FIFO		
5	USBCTRL_IRQ	11	DMA_IRQ_0	17	CLOCKS_IRQ	23	I2C0_IRQ		

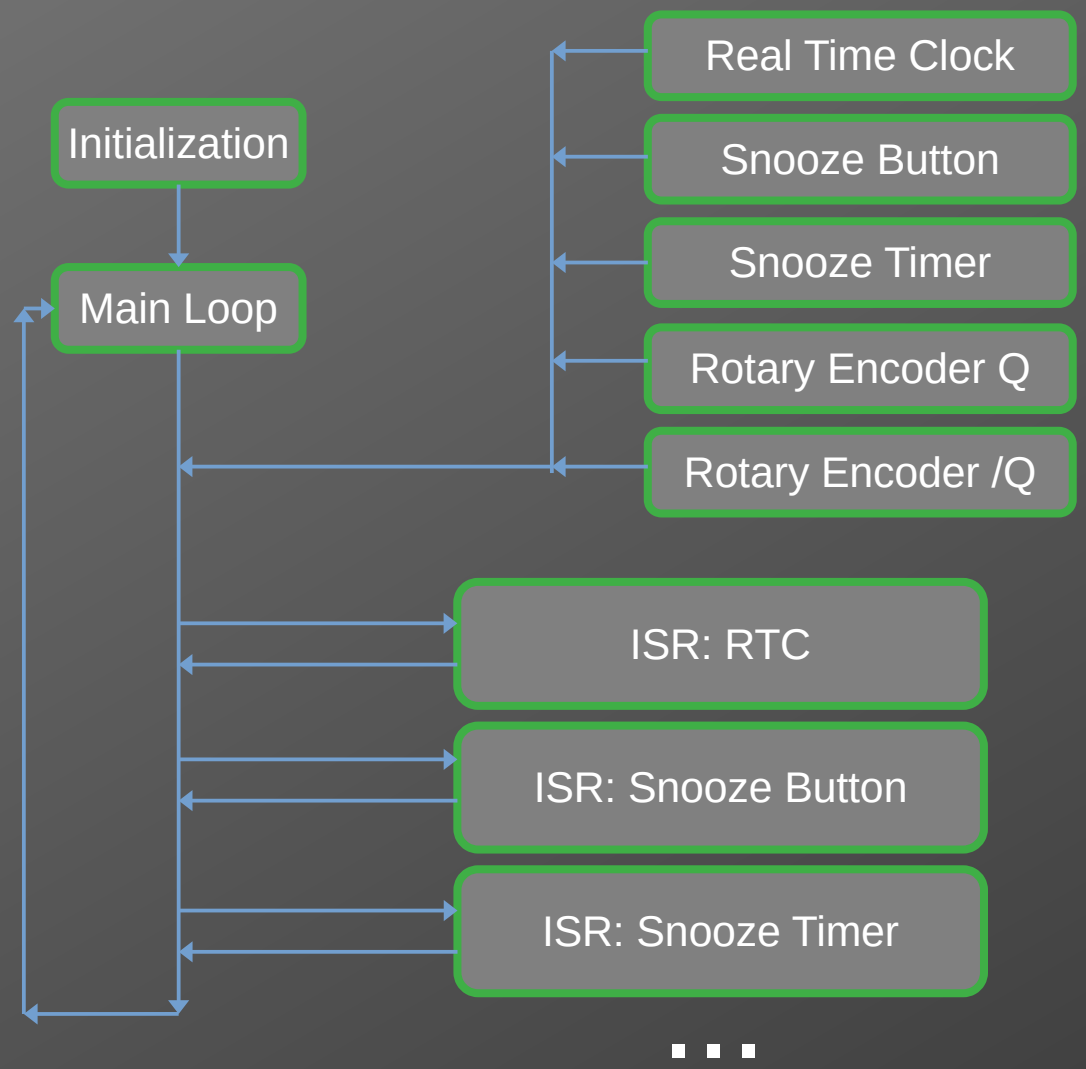
Some care has gone into arranging the RP2040 interrupt table to give a sensible default priority ordering, but individual interrupts can be raised or lowered in priority, using NVIC_IPR0 through NVIC_IPR7, to suit a particular use case.

ISRs Enable Event Driven Software

Can be a big “level-up” for hobbiests
- Code responds to events → “event driven”
- Response to events is very fast
- Loop can be less complex
- CPU utilization can be dramatically reduced



Event Driven Alarm Clock



“Simple” Button Press Counter

Works great… except for switch bounce

```
#include <Arduino.h>

const int BUTTON_PIN = 15;

volatile uint32_t pressCount = 0;
volatile bool updated = false;

// ISR
void handleButtonPress() {
    pressCount++;
    updated = true;
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    pinMode(BUTTON_PIN, INPUT_PULLUP);

    // Attach interrupt to FALLING edge (Button to GND)
    attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), handleButtonPress, FALLING);

    Serial.println("Counter Ready.");
}

void loop() {
    if (updated) {
        updated = false;
        Serial.print("Button Press Count: ");
        Serial.println(pressCount);
    }
}
```

“Debounced” Button Press Counter

Works great… except it triggers on both btn press and release due to release bounce

```
#include <Arduino.h>

const int BUTTON_PIN = 15;
const int LED_PIN = 25;

volatile uint32_t pressCount = 0;
volatile bool updated = false;

// Debounce variables
volatile uint32_t lastInterruptTime = 0;
const uint32_t debounceDelay = 100;

// The ISR
void handleButtonPress() {
    uint32_t interruptTime = millis();

    // If interrupts come faster than 200ms, assume it's a bounce and ignore
    if (interruptTime - lastInterruptTime > debounceDelay) {
        pressCount++;
        updated = true;
        digitalWrite(LED_PIN, !digitalRead(LED_PIN));

        lastInterruptTime = interruptTime;
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    // Attach interrupt to FALLING edge (Button to GND)
    attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), handleButtonPress, FALLING);

    Serial.println("Debounced Counter Ready.");
}

void loop() {
    if (updated) {
        updated = false;
        Serial.print("Clean Press Count: ");
        Serial.println(pressCount);
    }
}
```

“Debounced” Button Press Counter With State Check

Works great!

```c
volatile alarm_id_t debounce_alarm_id = 0;

// ISR Callback
int64_t alarm_callback(alarm_id_t id, void *user_data) {
    bool currentState = digitalRead(BUTTON_PIN);

    if (currentState != lastStableState) {
        if (currentState == LOW) { // Valid Press
            pressCount++;
            needsReport = true;
            sio_hw->gpio_togl = (1 << LED_PIN);
        }
        lastStableState = currentState;
    }

    debounce_alarm_id = 0; // Reset ID so we know no alarm is pending
    return 0;
}

// The GPIO ISR
void handle_gpio_event() {
    // If an alarm is already pending, cancel it so we can "restart" the clock
    if (debounce_alarm_id > 0) {
        cancel_alarm(debounce_alarm_id);
    }

    // Schedule the check for 50ms from now
    debounce_alarm_id = add_alarm_in_ms(50, alarm_callback, NULL, false);
}
```

Interrupt Code is !(Always Portable)

This compiles for Pi Pico But not for Arduino Nano

```
#include <Arduino.h>

const int LED_PIN = 25;
const int D14_PIN = 14;

volatile bool ledState = false;
bool repeating_timer_callback(struct repeating_timer *t) {
    digitalWrite(D14_PIN, HIGH);
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState);
    digitalWrite(D14_PIN, LOW);
    return true; // Return true to keep the timer running
}

struct repeating_timer timer;

void setup() {
    pinMode(LED_PIN, OUTPUT);
    pinMode(D14_PIN, OUTPUT);
    add_repeating_timer_ms(-500, repeating_timer_callback, NULL, &timer);
}

void loop() {

}
```

This compiles for Arduino Nano

```
#include <avr/io.h>
#include <avr/interrupt.h>

const int LED_PIN = 13;
const int D14_PIN = 14;

volatile bool ledState = false;

ISR(TIMER1_COMPA_vect) {
    digitalWrite(D14_PIN, HIGH); // Start profiling
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState);
    digitalWrite(D14_PIN, LOW);  // End profiling
}

void setup() {
    pinMode(LED_PIN, OUTPUT);
    pinMode(D14_PIN, OUTPUT);

    noInterrupts();           // Disable interrupts during configuration

    TCCR1A = 0;               // Set entire TCCR1A register to 0
    TCCR1B = 0;               // Same for TCCR1B
    TCNT1  = 0;               // Initialize counter value to 0

    OCR1A = 31249;
    TCCR1B |= (1 << WGM12);
    TCCR1B |= (1 << CS12);
    TIMSK1 |= (1 << OCIE1A);

    interrupts();             // Re-enable interrupts
}

void loop() {

}
```

Pin Change Code May be Portable

This compiles for both Pico and Nano

```
#include <Arduino.h>

// --- Board-Specific Pin Mapping ---
#if defined(ARDUINO_ARCH_RP2040) || defined(ARDUINO_ARCH_RP2350)
  const int STOP_BUTTON_PIN = 2; // GPIO 2 on Pico
  const int STATUS_LED = 25;     // Onboard LED
#else
  const int STOP_BUTTON_PIN = 2; // Digital Pin 2 (INT0) on Nano
  const int STATUS_LED = 13;     // Onboard LED
#endif

volatile bool systemSafe = true;

void handleEmergencyStop() {
    systemSafe = false;
    digitalWrite(STATUS_LED, LOW);
}

void setup() {
    Serial.begin(115200);
    pinMode(STATUS_LED, OUTPUT);
    digitalWrite(STATUS_LED, HIGH); // System starts "running"
    pinMode(STOP_BUTTON_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(STOP_BUTTON_PIN), handleEmergencyStop, FALLING);
    Serial.println("System Running. Press E-Stop to halt.");
}

void loop() {
    if (systemSafe) {
        Serial.println("Working...");
        delay(1000);
    } else {
        Serial.println("!!! EMERGENCY STOP ACTIVATED !!!");
        while (1) {
            // Stay here until the board is reset
        }
    }
}
```

ATMEGA 328P Pin Change Notes

11.1 Interrupt Vectors in ATmega328P

Table 11-1. Reset and Interrupt Vectors in ATmega328P

Vector No.	Program Address	Source	Interrupt Definition
1	0x0000	RESET	External pin, power-on reset, brown-out reset and watchdog system reset
2	0x002	INT0	External interrupt request 0
3	0x0004	INT1	External interrupt request 1
4	0x0006	PCINT0	Pin change interrupt request 0
5	0x0008	PCINT1	Pin change interrupt request 1
6	0x000A	PCINT2	Pin change interrupt request 2
7	0x000C	WDT	Watchdog time-out interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 compare match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 compare match B
10	0x0012	TIMER2 OVF	Timer/Counter2 overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 capture event
12	0x0016	TIMER1 COMPA	Timer/Counter1 compare match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 compare match B
14	0x001A	TIMER1 OVF	Timer/Counter1 overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 compare match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 compare match B
17	0x0020	TIMER0 OVF	Timer/Counter0 overflow
18	0x0022	SPI, STC	SPI serial transfer complete
19	0x0024	USART, RX	USART Rx complete
20	0x0026	USART, UDRE	USART, data register empty
21	0x0028	USART, TX	USART, Tx complete
22	0x002A	ADC	ADC conversion complete
23	0x002C	EE READY	EEPROM ready
24	0x002E	ANALOG COMP	Analog comparator
25	0x0030	TWI	2-wire serial interface
26	0x0032	SPM READY	Store program memory ready

INTx are (usually) faster than PCINTx

Both take 4 cycles to jump to/from the ISR but PCINTx usually requires code to determine which pin changed

INTx can be used for wake-up

ISR Best Practices

1. Keep ISRs short
→ Consider setting a flag in the ISR and handling the rest in the main loop
→ Direct port masking, or other lower level hardware rather than Arduino built-ins

2. Don’t use delay() as it may need interrupts being used
3. Don’t use printf() because it is slow and uses interrupts
4. Use the volatile keyword when sharing a variable between your ISR and your loop

How Fast is an ISR?

6.7.1 Interrupt Response Time

The interrupt execution response for all the enabled AVR® interrupts is four clock cycles minimum. After four clock cycles the program vector address for the actual interrupt handling routine is executed. During this four clock cycle period, the program counter is pushed onto the stack. The vector is normally a jump to the interrupt routine, and this jump takes three clock cycles. If an interrupt occurs during execution of a multi-cycle instruction, this instruction is completed before the interrupt is served. If an interrupt occurs when the MCU is in sleep mode, the interrupt execution response time is increased by four clock cycles. This increase comes in addition to the start-up time from the selected sleep mode.

A return from an interrupt handling routine takes four clock cycles. During these four clock cycles, the program counter (two bytes) is popped back from the stack, the stack pointer is incremented by two, and the I-bit in SREG is set.

1/16MHz = 62.5ns
4*62.5 + 4*62.5 = 500ns

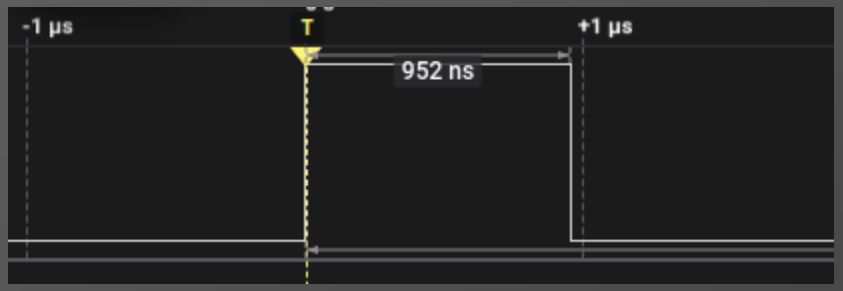
PINB = (1 << 5); // This toggles PB5 (Digital Pin 13)
1 clock cycle

562.5ns minimum

Tricks to speed up an ISR

Direct port masking is 50 times faster

```
volatile bool ledState = false;

bool repeating_timer_callback(struct repeating_timer *t) {
    digitalWrite(D14_PIN, HIGH);
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState);
    digitalWrite(D14_PIN, LOW);
    return true; // Return true to keep the timer running
}
```



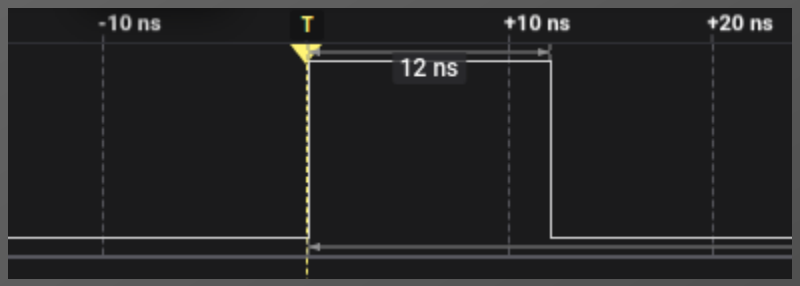

```
const int LED_PIN = 25;
const int D14_PIN = 14;

bool repeating_timer_callback(struct repeating_timer *t) {
    sio_hw->gpio_set = (1u << D14_PIN);
    sio_hw->gpio_togl = (1u << LED_PIN);
    sio_hw->gpio_clr = (1u << D14_PIN);
    return true;
}
```

150MHz → 6.667ns clock

Tricks to speed up an ISR

Set a flag and then deal with it in loop()

```
volatile bool flag = false;

bool repeating_timer_callback(struct repeating_timer *t) {
    sio_hw->gpio_set = (1u << D14_PIN);
    flag = true;
    sio_hw->gpio_clr = (1u << D14_PIN);
    return true;
}
```



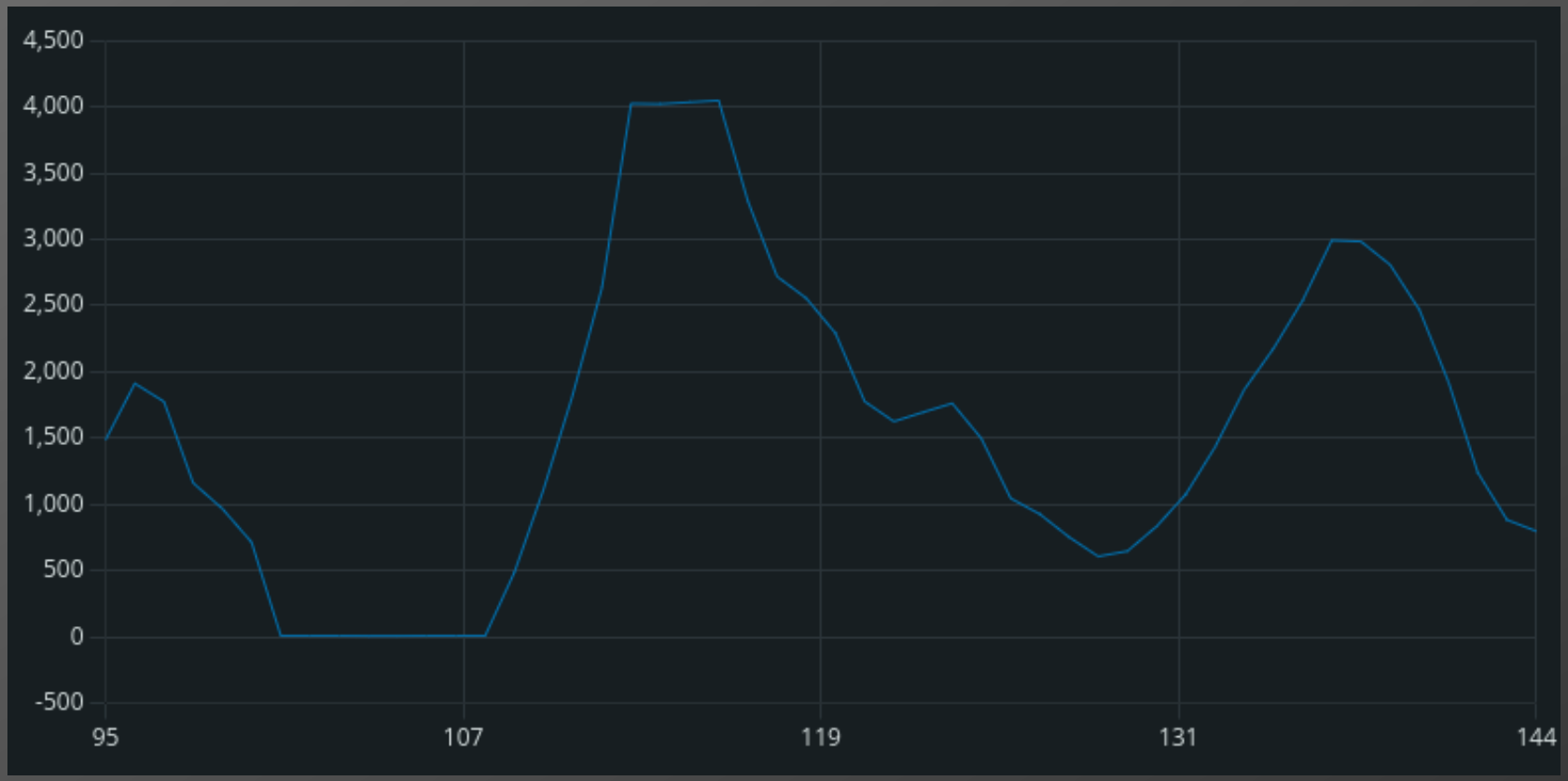
```
void loop() {
  if (flag) {
    sio_hw->gpio_set = (1u << D14_PIN);
    flag = false;
    uint16_t raw = adc_read();
    const float conversion_factor = 3.3f / 4096.0f;
    float voltage = raw * conversion_factor;
    Serial.print("Raw_Value:");
    Serial.print(raw);
    Serial.print(",");
    Serial.print("Voltage_V:");
    Serial.println(voltage, 3); // Print with 3 decimal places
    sio_hw->gpio_clr = (1u << D14_PIN);

  }
```

ISR provides precise timing

```
volatile bool flag = false;

bool repeating_timer_callback(struct repeating_timer *t) {
    flag = true;
    return true;
}
```

```
void loop() {
  if (flag) {
    flag = false;
    sio_hw->gpio_set = (1u << D14_PIN);
    // random delay, 1-25ms
    delay(random(1, 26));
    sio_hw->gpio_clr = (1u << D14_PIN);
  }
```

See ADC Example (100ms Sampling)

```
// Volatile variables for ISR-to-Main communication
volatile uint16_t adc_value = 0;
volatile bool data_ready = false;

// This runs in the background every 100ms
bool timer_callback(struct repeating_timer *t) {
    adc_value = adc_read();
    data_ready = true;

    return true; // Keep the timer running
}
```

```
void loop() {
    // 3. Main loop handles the "slow" Serial output
    if (data_ready) {
        data_ready = false;

        // Output for Serial Plotter
        Serial.println(adc_value);
    }
}
```